NTNU | Faculty of Information Technology and Electrical Engineering
Department of Electronic Systems

Specialization Projects

Specialization Project fall 2026

Hyperspectral Data Compression Using Deep Learning

Author:
Peder Ingmar Dahl

Supervisor:
Prof. Milica Orlandic

table of contents

1 Introduction **2**

1.1 Current project plan 2

2 Theory **3**

3 Method **4**

4 Results **5**

5 Discussion **6**

6 Conclusion **7**

References **8**

Appendixes **9**

Abstract

1 Introduction

1.1 Current project plan

I should decide on a model or a type of model to compress for efficient inference on a specific hardware. The goal is to save both space, processing time, power ...

Options for models to compress: - 1D-Justo-LiuNet (1D CNN for land-sea-cloud classification onboard hypso 2) - any hyperspectral data compression models?

Options for specific hardware: - hypso 2: AMD/Xilinx Zynq-7000 - SoC = dual/quad ARM Cortex-A9 hard processor + programmable logic - hypso 3: Xilinx/AMD Zynq UltraScale+ - targeted to use the Kria K26 SoM. Vitis AI DPU support and maybe sub-8-bit options

When benchmarking I will do some testing of inference on gpus and cpus, but I also want to test inference with fpga in the loop. For this i could use Vitis AI for fast deployment on Xilinx devices. Altough Vitis HLS AI or similar i likely to be used when hardware acceleration for onboard models is developed by smallsat. F.ex 1D-Justo-LiuNet is deployed ombord hypso2, but runs on the cpus. Another student projects is to look into using hls for AI acceleration/matrix operations.

The software pipeline i will be using for model compression is likely: Keras/PyTorch -¿ ONNX -¿ Vitis AI DPU -¿ .xmodel on K26.

I will start by looking into model quantization. Converting down to INT8 activations is a safe start and will save much memory and runtime since the fpgas DPU blocks is guaranteed to support it and the biggest speedup comes from using these blocks. A possibility is to try INT4 per-channel-scaled weights, but i need to verify DPU support. Mixed precisions for different layers might be possible. Some differnet ways to quantize accoress the model: - per tensor (simpler) vs. a per-channel/per-group (better accuarcy) quantization - symmetric (simpler) vs asymmeric (better but less support) quantization - a trick: calibrate asymmetric, fold zero-point to symmetric for the DPU at compile.

Quantization aware training (QAT) is also a technique i can look into.

Next I will look into Pruning. Unstructured pruning (zeroing individual weights) is simple but not effective on fpga i read. Structured pruning is better.

Taking a look at Pruning + quantization interplay is interessting here. Pruning changes distribution. Aggressive pruning can hurt quantization accuracy since sparser/irregular weight ranges are harder to quantize, so these need to be jointly tuned, not independently.

Knowledge Distillation is about using the full model as a teacher to the compressed model (student) to recover some accuracy. It can be applied sequentially during quantization / quantization aware training and pruning. Knowledge distillation could be a good way to tune quantization and pruning jointly.

2 Theory

3 Method

4 Results

5 Discussion

6 Conclusion

References

Appendices